

Agentes inteligentes en entornos complejos: el caso de Catan

Herramientas y Aplicaciones de la IA

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Universitat Politècnica de València

- En esta sesión aprenderemos a diseñar agentes capaces de tomar decisiones en entornos complejos, usando **Catan** como banco de pruebas y **PyCatan** como plataforma experimental. El objetivo: pasar de decisiones aleatorias a políticas heurísticas que mejoren el rendimiento del agente.



Objetivos

Al finalizar deberíais ser capaces de:

01

Diseño de agentes

Comprender cómo se diseñan agentes inteligentes en entornos complejos

03

Catan y PyCatan

Conocer el juego desde la perspectiva de la IA y el simulador PyCatan

02

Simulación como herramienta

Entender el papel de la simulación en la investigación en IA

04

Implementación y evaluación

Implementar un agente heurístico básico y evaluar su rendimiento mediante simulación



Diseñar agentes es un **proceso iterativo** de diseño, experimentación y evaluación.

REPASO SMA

¿Qué es un agente?

Un **agente** es un sistema que percibe su entorno y actúa sobre él para alcanzar ciertos objetivos.

Formalmente, el agente implementa una **política de decisión**: π : estado $\rightarrow$ acción

Percepción

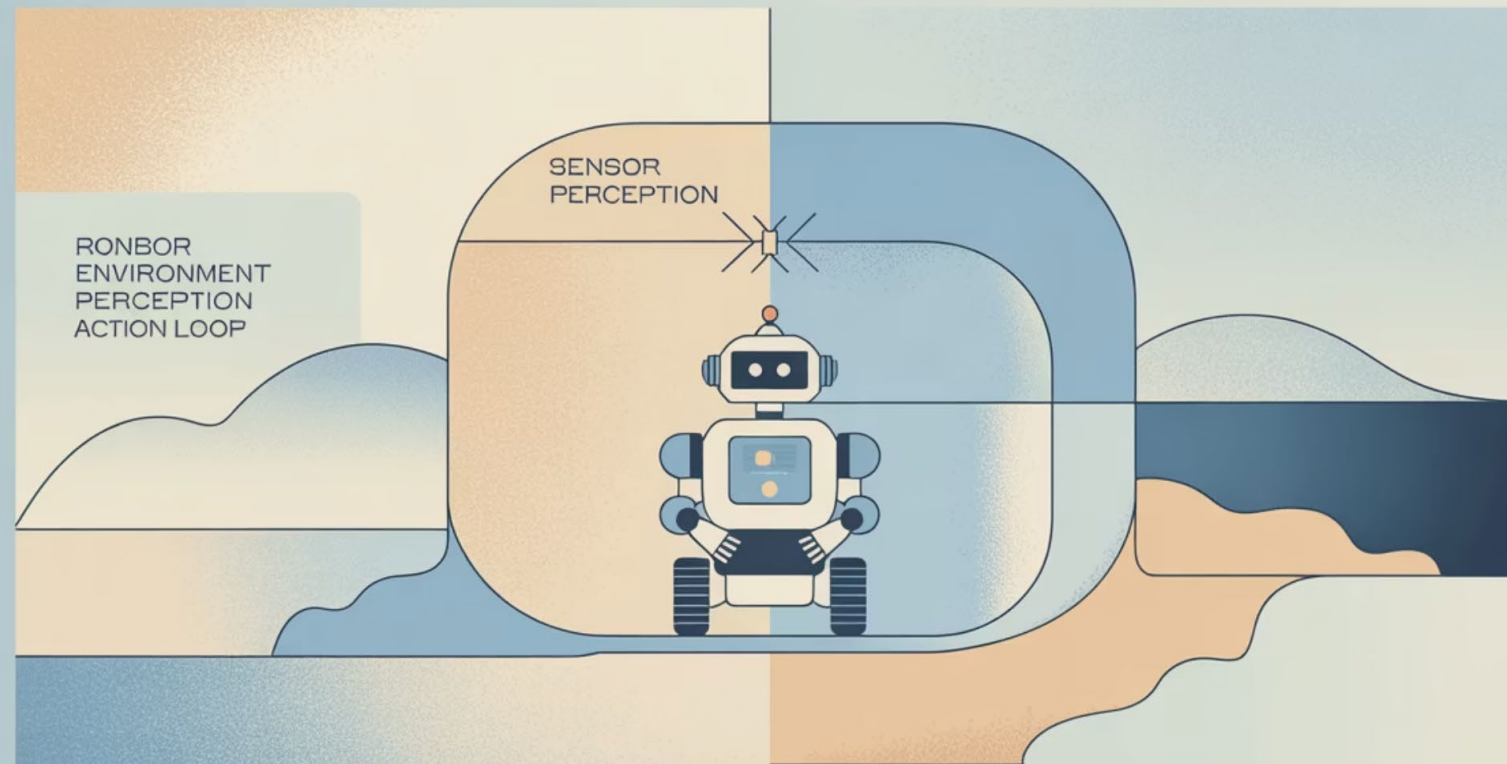
Obtiene información del entorno

Decisión

Selecciona una acción

Acción

Modifica el entorno



Propiedades de los agentes inteligentes



Autonomía

Toma decisiones sin intervención externa



Reactividad

Responde a cambios en el entorno



Proactividad

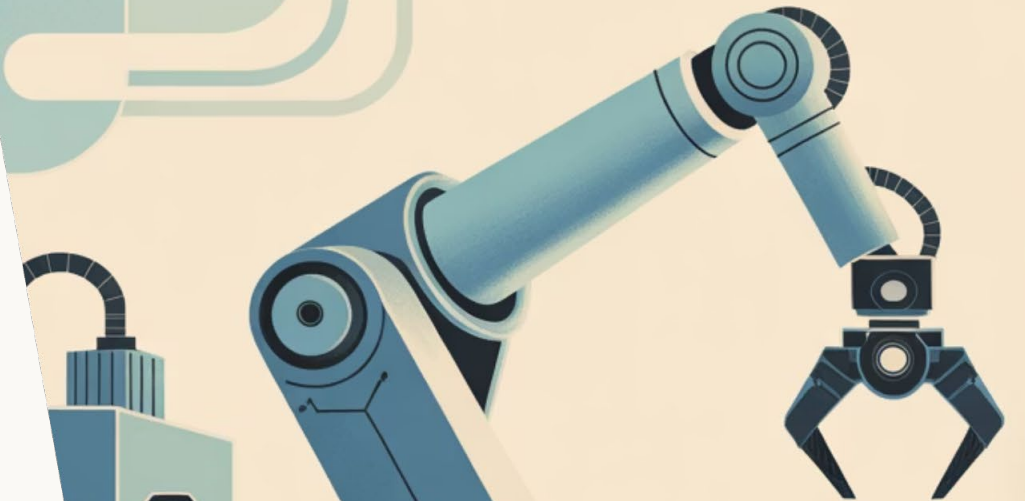
Persigue objetivos a largo plazo



Interacción social

Interactúa con otros agentes o humanos

Ejemplos reales: robots autónomos, asistentes virtuales, sistemas de trading, agentes en videojuegos



Tipos de agentes

La literatura clásica de IA distingue varios tipos según su capacidad de razonamiento:

1

Agentes reactivos

Responden al estado actual

2

Basados en modelo

Mantienen representación del entorno

3

Basados en objetivos

Eligen acciones para alcanzar metas

4

Basados en utilidad

Maximizan una función de utilidad

5

Agentes que aprenden

Mejoran con la experiencia

📌 Comenzaremos con **agentes heurísticos simples**, el punto de partida más natural antes de introducir aprendizaje o decisiones complejas

CONCEPTO CENTRAL

Políticas de decisión

Un agente implementa una política $\pi(s) \rightarrow a$, donde s es el estado del entorno y a la acción seleccionada.

La política puede construirse con distintos enfoques, de menor a mayor complejidad:

📄 En esta sesión nos centraremos en **heurísticas**.

→ Reglas

Condición → acción fija

→ Optimización

Búsqueda del óptimo

¿Qué es una heurística?

Una heurística es una regla de decisión práctica que no garantiza el óptimo pero ofrece soluciones razonables de forma eficiente.

Ejemplo: *"Si puedo construir una ciudad, construyo; si no, construyo carretera hacia el mejor hexágono disponible."*

→ Heurísticas

Criterios de evaluación rápidos

→ Aprendizaje automático / LLM

Políticas aprendidas de datos

Simulación en Inteligencia Artificial

Los sistemas complejos rara vez admiten soluciones matemáticas cerradas. La **simulación** es la alternativa experimental.



Experimentar estrategias

Probar distintas políticas sin costes reales



Miles de experimentos

Ejecutar partidas masivamente para obtener estadísticas fiables



Condiciones controladas

Aislar variables y reproducir escenarios exactos



Análisis estadístico

Comparar agentes con métricas cuantitativas robustas



Simulación: áreas de aplicación



Movilidad

Tráfico urbano y transporte público



Economía

Mercados financieros y comportamiento de consumidores



Logística

Gestión de almacenes y rutas de reparto



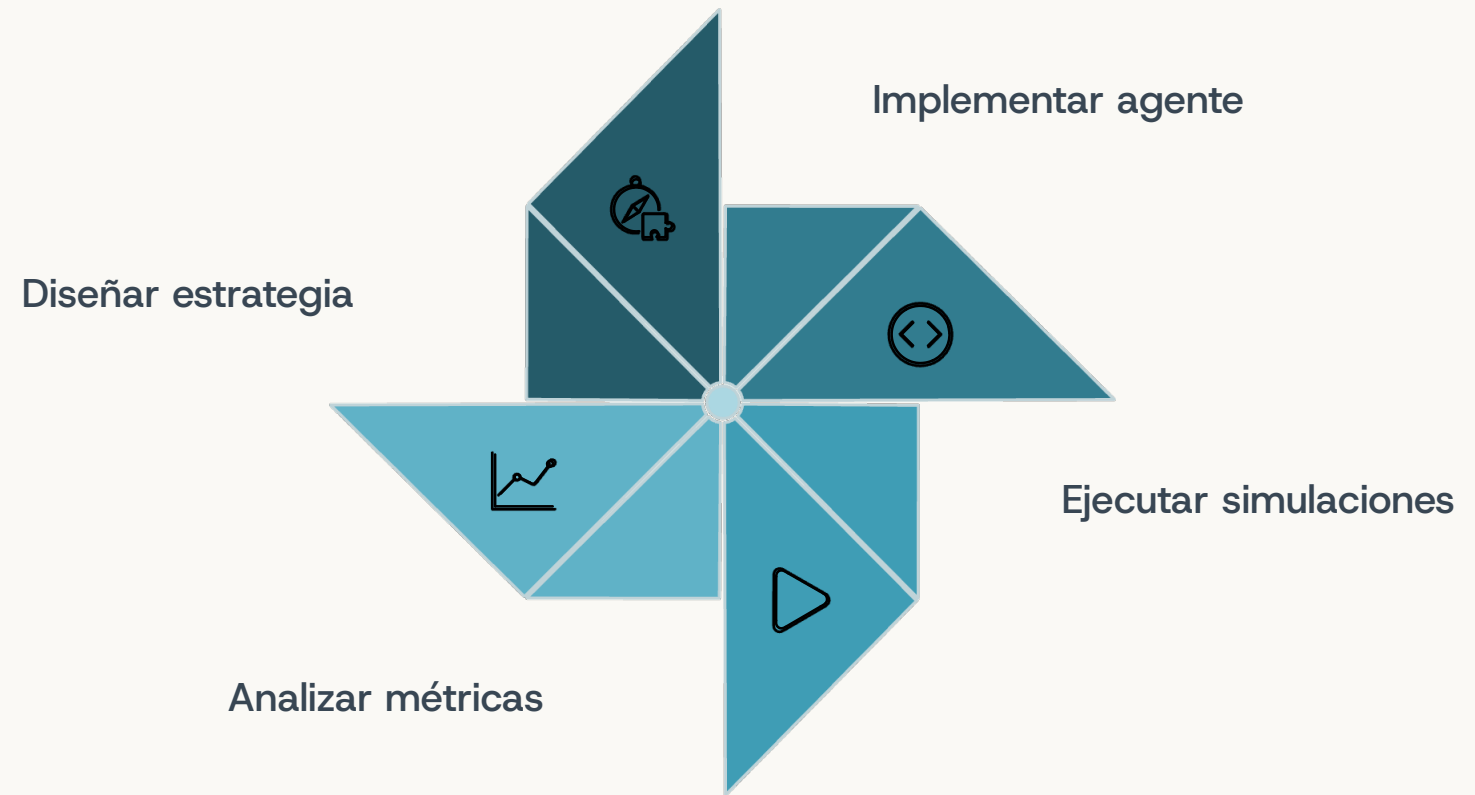
Videojuegos

Diseño y evaluación de agentes estratégicos

Evaluación empírica de agentes

El desarrollo de un agente sigue un **ciclo experimental iterativo**. La métrica más común depende del tipo de agente, pero suele medir *utilidad, beneficio, coste*, etc.

Este enfoque es estándar tanto en investigación académica como en aplicaciones industriales. Cuantas más simulaciones y escenarios distintos, más fiable es la estimación del rendimiento real.





Juegos como banco de pruebas de IA

Grandes avances en IA han surgido del estudio de juegos. ¿Por qué son tan valiosos?

Reglas claras

Entorno bien definido y reproducible

Decisiones complejas

Espacios de búsqueda enormes y estrategias variadas

Interacción multiagente

Competición y cooperación entre jugadores

Simulación masiva

Miles de partidas en poco tiempo

Hitos históricos

- **Ajedrez** — Deep Blue vs. Kasparov (1997)
- **Go** — AlphaGo vs. Lee Sedol (2016)
- **Poker** — Libratus (2017)
- **StarCraft II** — AlphaStar (2019)
- **Dota 2** — OpenAI Five (2019)

En esta práctica: **Catan + PyCatan** 🎲

Sistemas Multiagente

Muchos problemas del mundo real implican múltiples agentes que interactúan entre sí. Un **sistema multiagente (MAS)** está formado por varios agentes autónomos que comparten un entorno y se influyen mutuamente.

¿Qué hacen los agentes?

- Deciden de forma autónoma
- Comparten un entorno común
- Cooperan o compiten
- Influyen en los demás

Ejemplos reales



Mercados financieros



Tráfico urbano



Logística distribuida



Videojuegos
multijugador



Características de los Sistemas Multiagente

El comportamiento global de un MAS es difícil de predecir precisamente porque cada agente reacciona a los demás. Estas cuatro propiedades definen su complejidad inherente.



Interdependencia

Las decisiones de un agente afectan directamente a los resultados de los demás.



Competencia

Los agentes pueden tener objetivos opuestos e irreconciliables.



Cooperación

En otros casos, los agentes colaboran para alcanzar metas comunes.



Información Parcial

Ningún agente conoce completamente el estado del sistema global.

Propiedades de los Entornos de Decisión

En IA, los entornos se caracterizan a lo largo de varias dimensiones clave. Comprender estas dimensiones es esencial para elegir el tipo de agente más adecuado.

Determinista vs. Estocástico

¿Las acciones tienen resultados predecibles o interviene el azar?

Estático vs. Dinámico

¿El entorno cambia mientras el agente está tomando su decisión?

1

2

3

4

Observable vs. Parcialmente Observable

¿El agente conoce el estado completo del entorno en cada momento?

Individual vs. Multiagente

¿Hay otros agentes activos tomando decisiones simultáneamente?

Espacio de Decisiones en Catan

A lo largo de una partida, el agente se enfrenta a una gran variedad de tipos de decisión. Cada una puede implementarse con estrategias diferentes y requiere una lógica de evaluación propia.



Colocación inicial

Selección de los dos asentamientos y carreteras de inicio.



Mejora a ciudades

Doblar la producción de recursos en posiciones clave.



Cartas de desarrollo

Uso estratégico de caballeros, carreteras y puntos de victoria.



Carreteras y asentamientos

Expansión táctica por el tablero para ganar territorio.



Comercio

Intercambios con jugadores o con el banco/puertos según el contexto.



Movimiento del ladrón

Bloquear rivales y redistribuir recursos del tablero.

¿Por qué Catan es interesante para la IA?

Catan combina múltiples dimensiones de complejidad que lo convierten en un banco de pruebas ideal para agentes inteligentes.



Incertidumbre

Los dados introducen aleatoriedad estructural en cada turno



Gestión de recursos

Equilibrar cinco tipos de recursos con demanda variable



Planificación

Decisiones secuenciales con horizonte temporal largo



Multiagente

Negociación y competencia simultánea entre jugadores

Objetivo del juego

10

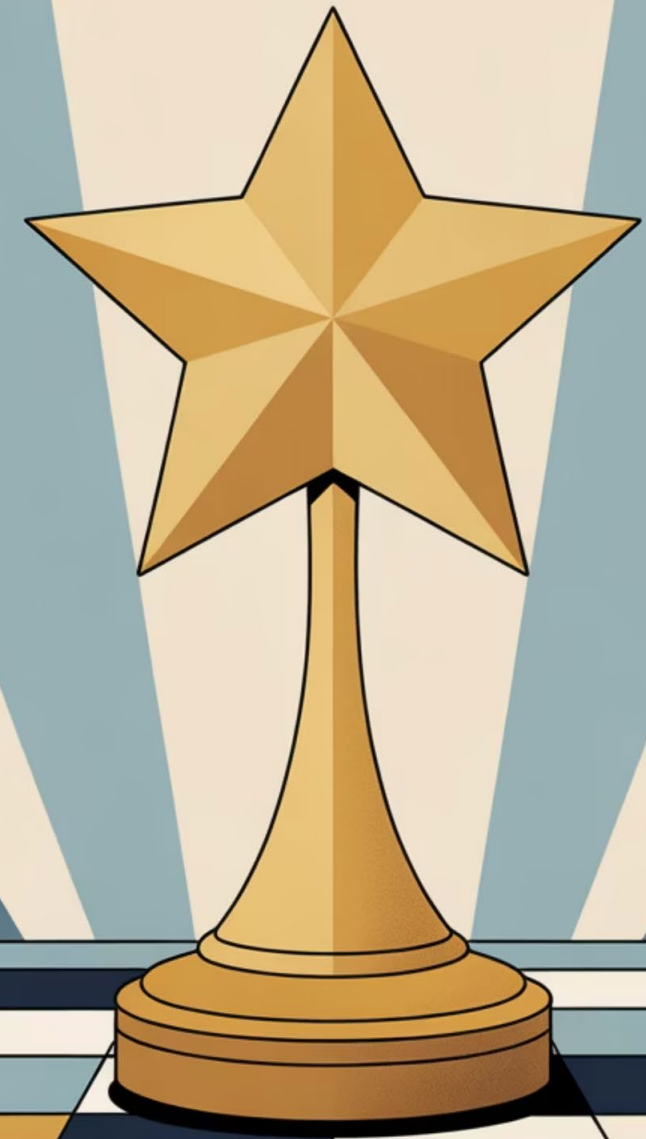
Puntos de victoria

Meta para ganar la partida

¿Cómo se consiguen?

- Asentamientos (1 punto cada uno)
- Ciudades (2 puntos cada una)
- Cartas de desarrollo especiales
- Logros: ejército más largo y camino más largo

El juego finaliza en el momento en que un jugador alcanza los 10 puntos durante su turno.



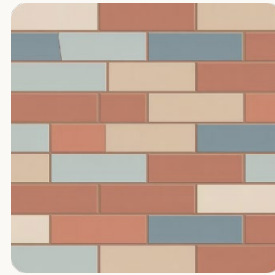
Los cinco recursos de Catan

Toda construcción y desarrollo en Catan depende de la combinación adecuada de estos recursos fundamentales.



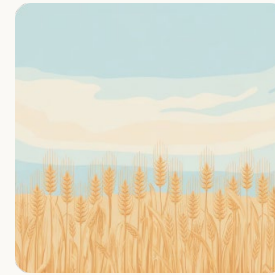
Madera

Construir carreteras y asentamientos



Ladrillo

Construir carreteras y asentamientos



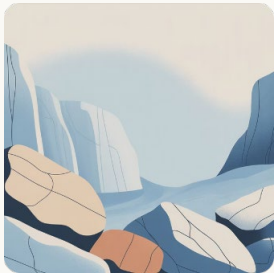
Trigo

Ciudades y cartas de desarrollo



Lana/Oveja

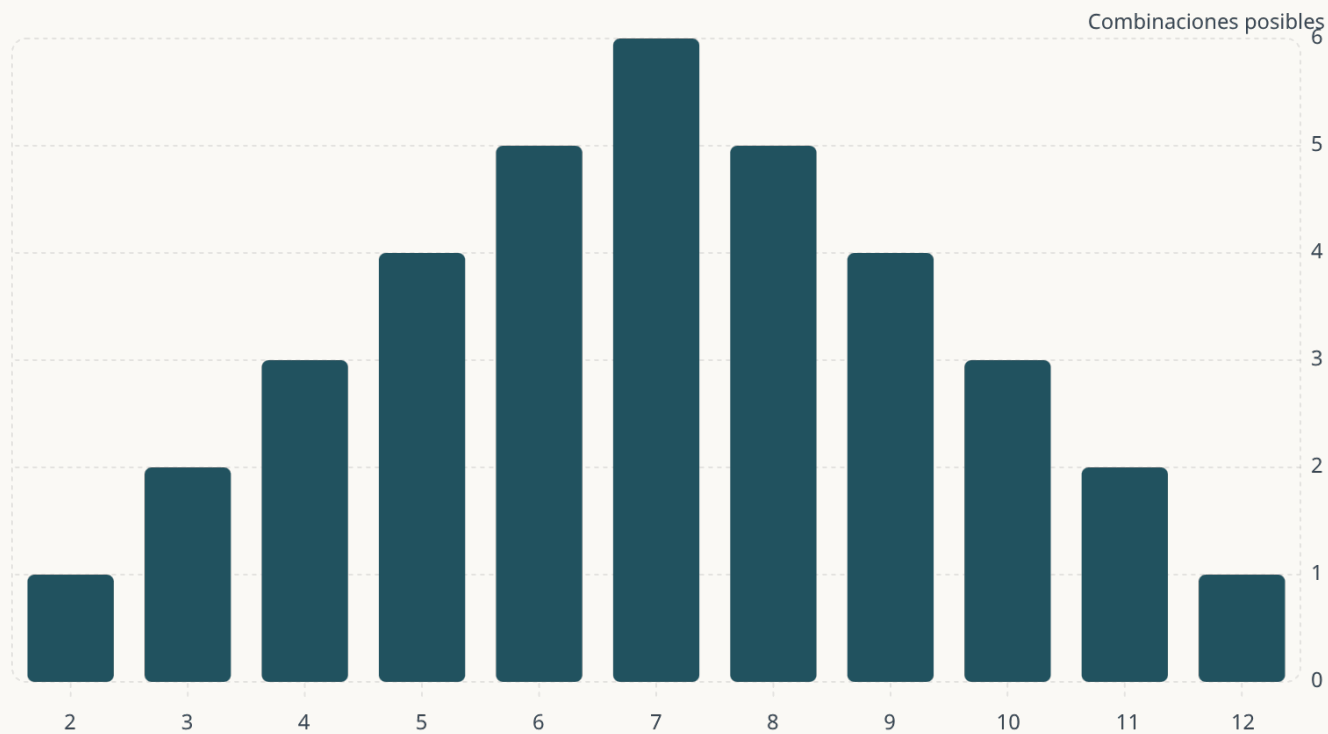
Asentamientos y cartas de desarrollo



Mineral

Ciudades y cartas de desarrollo

Producción de recursos: probabilidades



Implicaciones estratégicas

La distribución de probabilidades en dos dados sigue una curva simétrica. Colocarse en casillas con números **6 y 8** maximiza la producción esperada.

- **6 y 8**: 5 combinaciones de 36
- **5 y 9**: 4 combinaciones de 36
- **7**: activa el ladrón (6/36)

Esta asimetría es clave para el diseño de agentes de colocación inicial.



Construcciones y sus costes



Carretera

Coste: **1 madera + 1 ladrillo** Expande la red de conexiones y es necesaria para llegar a nuevas posiciones. El camino más largo otorga 2 puntos extra.



Asentamiento

Coste: **1 madera + 1 ladrillo + 1 trigo + 1 oveja** Genera recursos en cada producción y vale **1 punto de victoria**.



Ciudad

Coste: **2 trigo + 3 mineral** Mejora un asentamiento existente, duplicando su producción. Vale **2 puntos de victoria**.



Carta de desarrollo

Coste: **1 oveja + 1 trigo + 1 mineral** Proporciona habilidades especiales, puntos ocultos o soldados para el ejército más numeroso.



An illustration of a thief wearing a dark suit and a fedora, leaning over a chessboard on a blue table. The thief is looking down at the board, which has several chess pieces on it. The background is a simple, stylized room with a light blue wall and a tan wall.

El ladrón: mecánica de interrupción

La tirada de **7** es la más probable del juego y desencadena la mecánica de mayor impacto estratégico.

01

Descarte obligatorio

Todo jugador con **más de 7 cartas** en mano debe descartar la mitad, generando pérdidas significativas para quienes acumulan recursos.

02

Movimiento del ladrón

El jugador activo mueve la ficha del ladrón a cualquier casilla del tablero, **bloqueando su producción** hasta que sea desplazado.

03

Robo selectivo

Se roba al azar una carta de recurso a un jugador que tenga un asentamiento o ciudad **adyacente a la casilla** elegida.

Comercio: el componente negociador

Comercio entre jugadores

Negociación directa y libre durante el turno del jugador activo. Permite intercambios asimétricos y alianzas tácticas.

Comercio con el banco

Ratio estándar de **4:1**, o **3:1 / 2:1** si se tiene acceso a puertos especiales en la costa del tablero.

Relevancia para la IA

El comercio entre jugadores introduce el componente de **toma de decisiones bajo incertidumbre social**: el agente debe estimar intenciones, evaluar propuestas y decidir cuándo negociar.

Los puertos costeros son activos estratégicos que deben considerarse desde la colocación inicial para optimizar la eficiencia del intercambio.



CASO DE ESTUDIO

Clasificación del Entorno de Catan

Desde la perspectiva de la IA, Catan es un entorno **mucho más complejo** que un problema de optimización clásico. Combina cuatro propiedades que lo hacen un banco de pruebas excelente.



Estocástico

Los dados introducen incertidumbre en cada turno.



Multiagente Competitivo

Varios jugadores compiten por alcanzar los 10 puntos.



Parcialmente Observable

Las cartas de otros jugadores permanecen ocultas.



Secuencial

Cada decisión condiciona los estados futuros del juego.



Las carreteras abren o cierran rutas de expansión.

Introducción a PyCatan

PyCatan (desarrollado por Adrián Heras: <https://github.com/jaumejordan/PyCatan>) es un simulador de código abierto diseñado específicamente para la investigación en IA aplicada a Catan.

Investigación en IA

Entorno estandarizado para comparar algoritmos y estrategias de forma reproducible

Desarrollo de agentes

Interfaz clara para implementar agentes personalizados con distintos niveles de complejidad

Experimentación

Ejecuta partidas completas de forma automatizada entre múltiples agentes simultáneamente



Arquitectura del sistema PyCatan

GameDirector

Orquesta la partida y consulta a los agentes

Agents

Implementan la lógica de decisión de los jugadores

Board

Representa el tablero y el estado geográfico

GameState

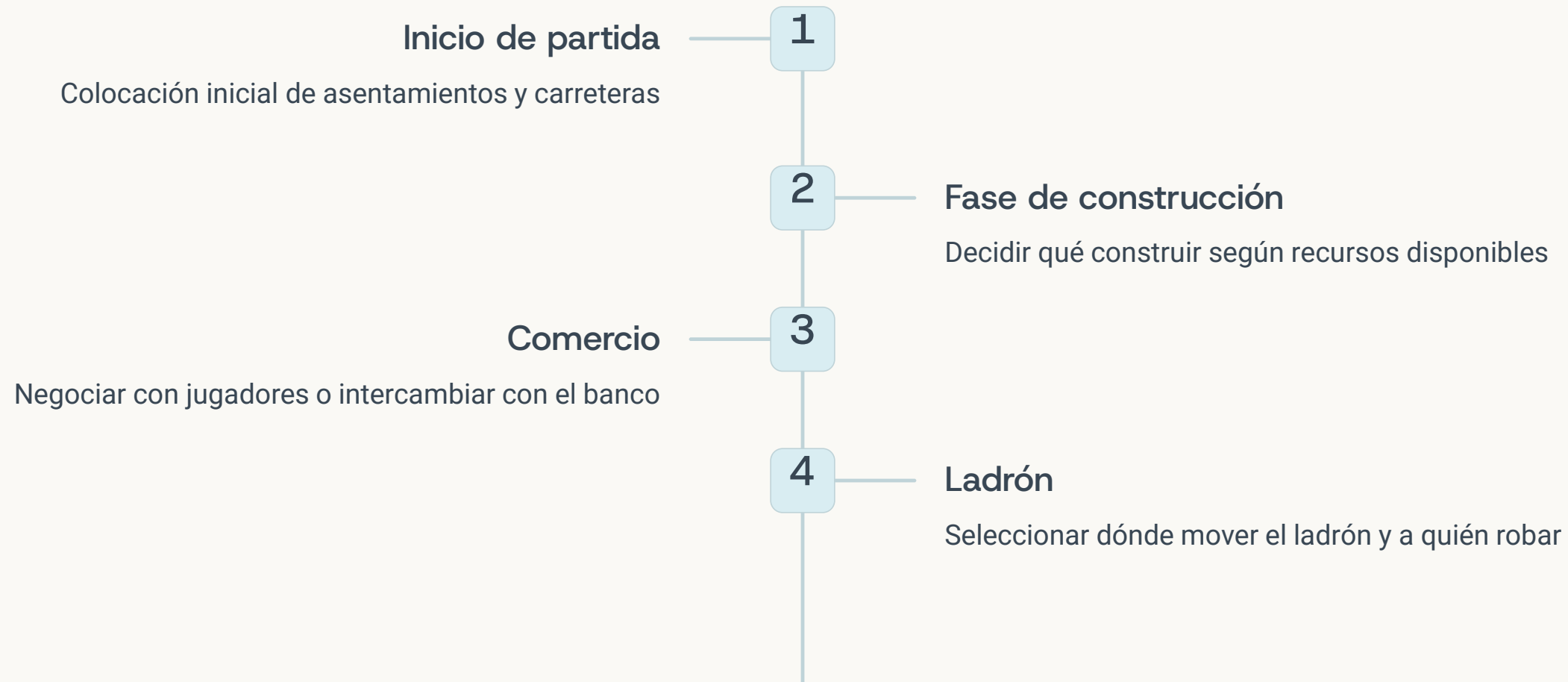
Almacena el estado completo de la partida



El **GameDirector** actúa como árbitro central: gestiona el flujo de la partida, aplica las reglas y lanza consultas a los agentes en cada momento de decisión, garantizando la coherencia del estado global.

Ciclo de decisión del agente

El agente interactúa con el sistema respondiendo a **eventos bien definidos**, cada uno implementado como un método de la clase agente.



❏ Cada método recibe el **GameState** completo como entrada y debe devolver una acción válida según las reglas del juego. El diseño de estos métodos define la inteligencia del agente.

Métodos principales de un agente

Un agente en PyCatan implementa métodos clave que responden a cada fase del juego. Cada método recibe el **estado actual** del tablero y devuelve una **acción válida**.



on_game_start()

Colocación inicial de asentamientos y carreteras



on_build_phase()

Decisión de construcción en cada turno



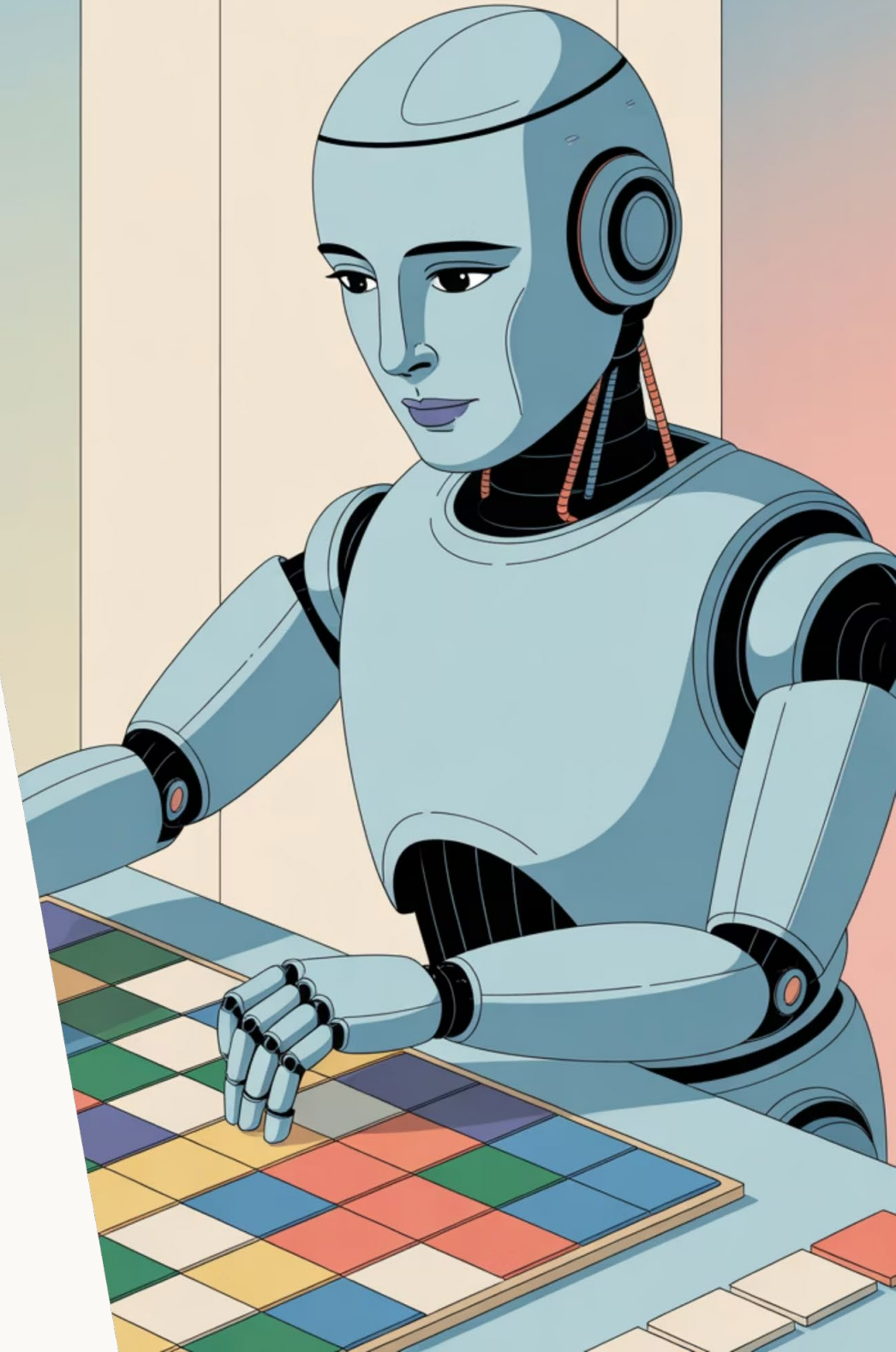
on_trade_offer()

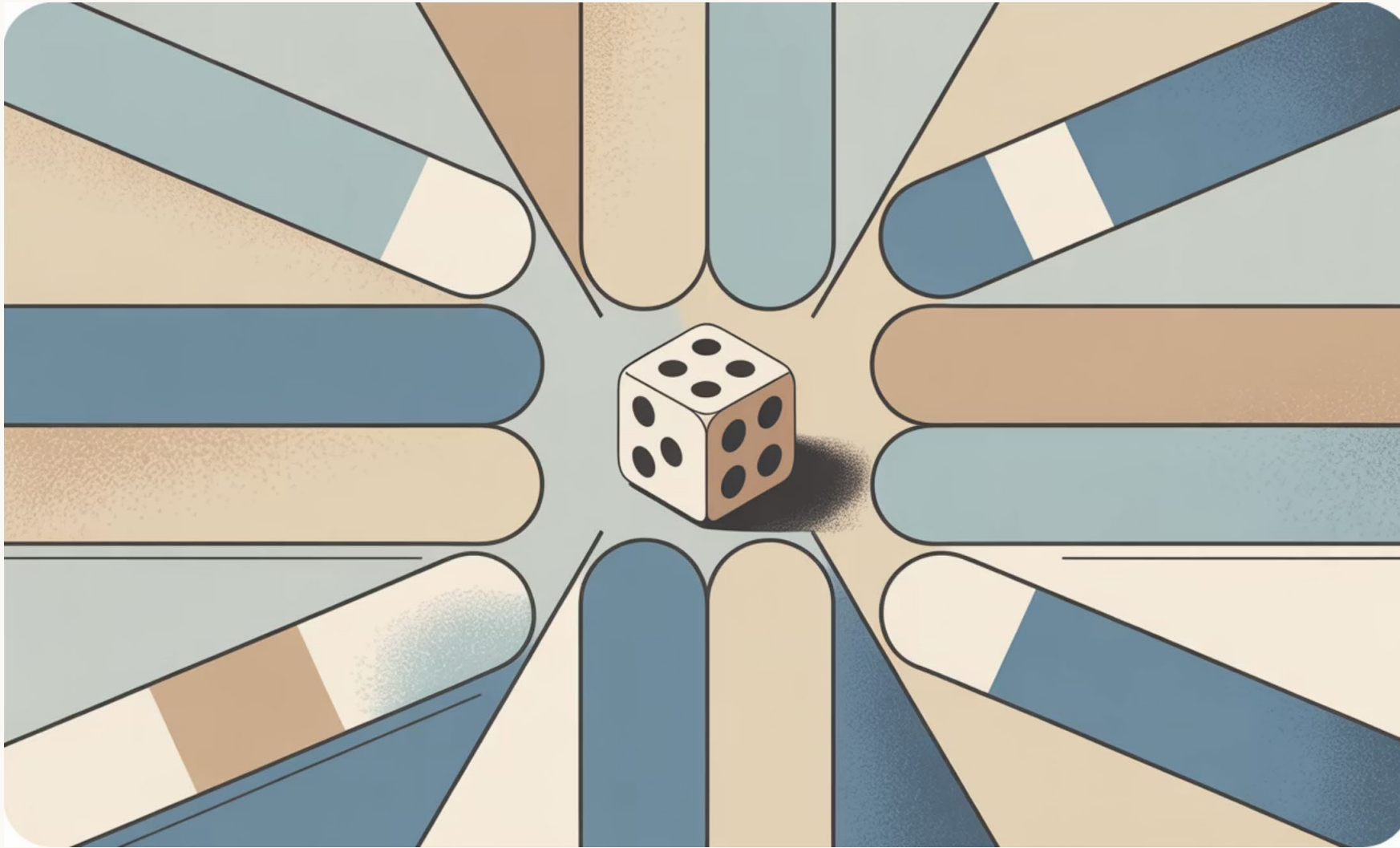
Aceptar o rechazar intercambios de recursos



on_moving_thief()

Elegir dónde mover al ladrón





PUNTO DE PARTIDA

Primer paso: usar RandomAgent

El **RandomAgent** toma decisiones completamente al azar. Es el **baseline** mínimo contra el que mediremos todo progreso.

→ **Empezar con RandomAgent**
Implementación sencilla y lista para usar

→ **Identificar decisiones clave**
¿Dónde ayuda más una regla inteligente?

→ **Sustituir por heurísticas**
Reemplazar aleatoriedad por reglas prácticas

¿Qué es una heurística?

Una heurística es una **regla práctica** que guía la toma de decisiones sin garantizar el resultado óptimo, pero produciendo resultados razonables de forma eficiente.



Si puedo construir ciudad

→ **Construir ciudad** (mayor retorno de recursos)



Si no puedo construir ciudad

→ **Intentar construir asentamiento**



Clave

No son perfectas, pero son **rápidas, simples y efectivas** como primer nivel de inteligencia

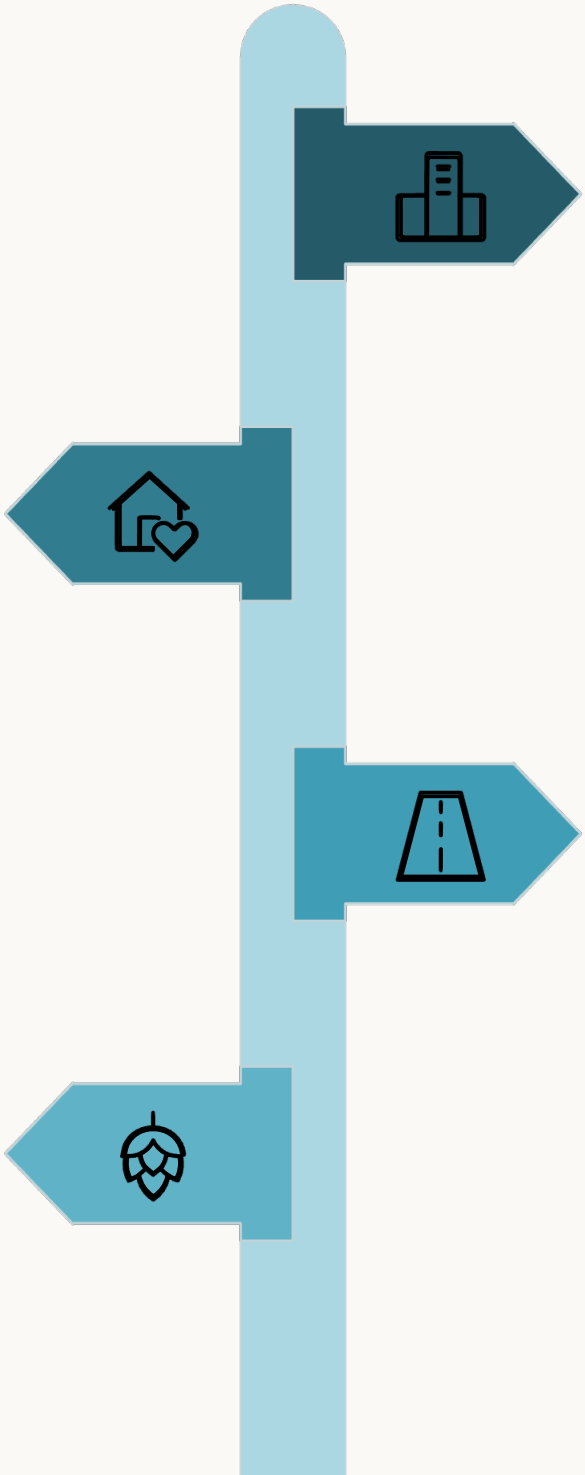


Heurística para la fase de construcción

Una política de decisión ordenada por **prioridad descendente** ya supera significativamente al agente aleatorio:

Construir asentamiento
Si no hubo ciudad, levantar asentamiento.

Pasar turno
Si nada es posible, terminar turno.



Construir ciudad
Priorizar ciudad si es posible.

Construir camino
Si no se puede construir otra, poner camino.

Heurística de colocación inicial

La posición inicial es uno de los factores más determinantes en Catan. Una mala colocación puede condenar la partida desde el inicio.

- ❏ **Estrategia básica:** Maximizar la producción esperada de recursos priorizando casillas con números **6 y 8**, los de mayor probabilidad de salir con dos dados.

También conviene diversificar los tipos de recurso obtenidos para no depender de uno solo.





Heurística para el ladrón

Mover al ladrón de forma inteligente puede frenar al rival más peligroso y cambiar el curso de la partida.

Casillas con alta producción

Números 6 u 8 → mayor impacto al bloquear

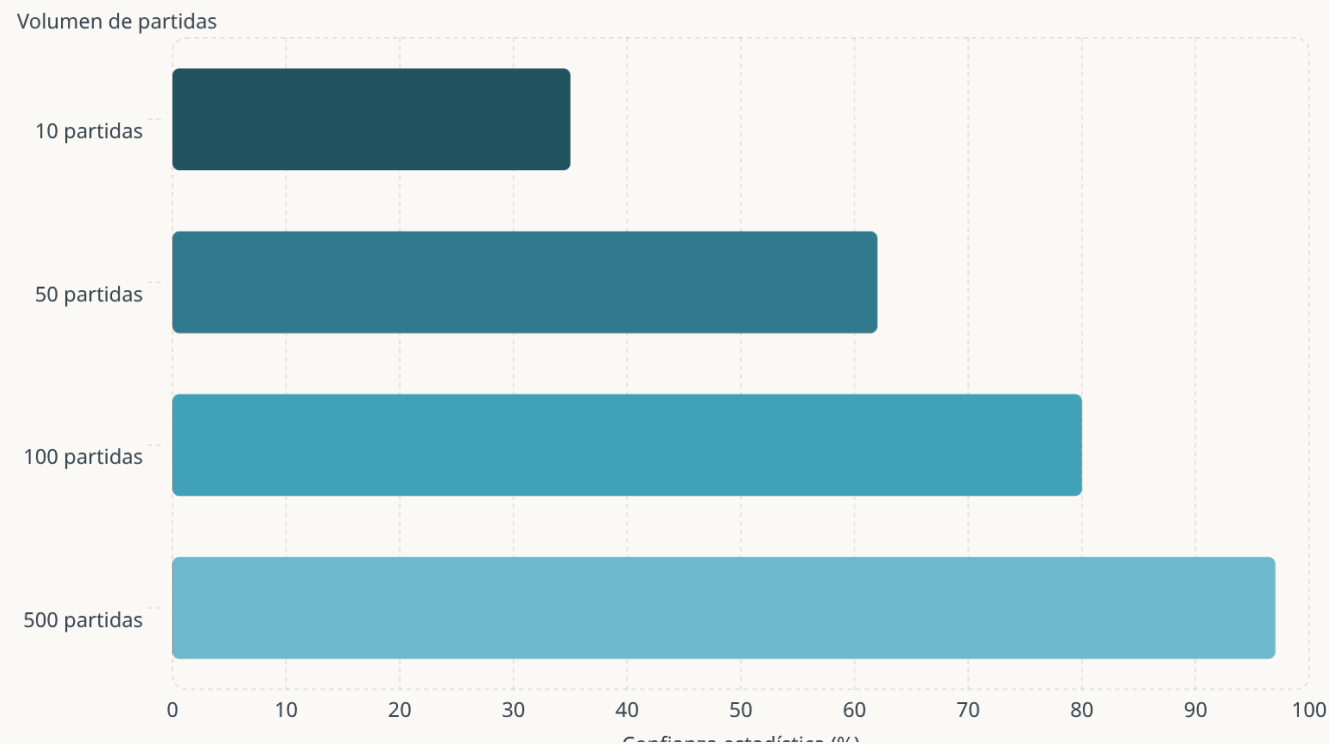
Del rival más fuerte

Reducir la producción del jugador líder en puntos



Efecto combinado

Retrasa al oponente y nivela la competición



EVALUACIÓN

Evaluación mediante simulación

Para medir el rendimiento real ejecutaremos **múltiples partidas** contra el **RandomAgent**.

- Mínimo recomendado: **100 partidas**
- Evaluación robusta: **500 partidas**
- A mayor volumen, mayor **fiabilidad estadística**



Pocos juegos pueden dar resultados engañosos.
Más simulaciones = más certeza.



Métricas de rendimiento

Estas métricas permiten **comparar estrategias** objetivamente y guiar las iteraciones de mejora.



Ratio de victorias

Porcentaje de partidas ganadas frente al total



Puntos medios

Promedio de puntos de victoria obtenidos por partida



Posición media

Clasificación final promedio entre los jugadores



Duración

Número medio de turnos por partida

Ciclo Completo de Desarrollo de Agentes

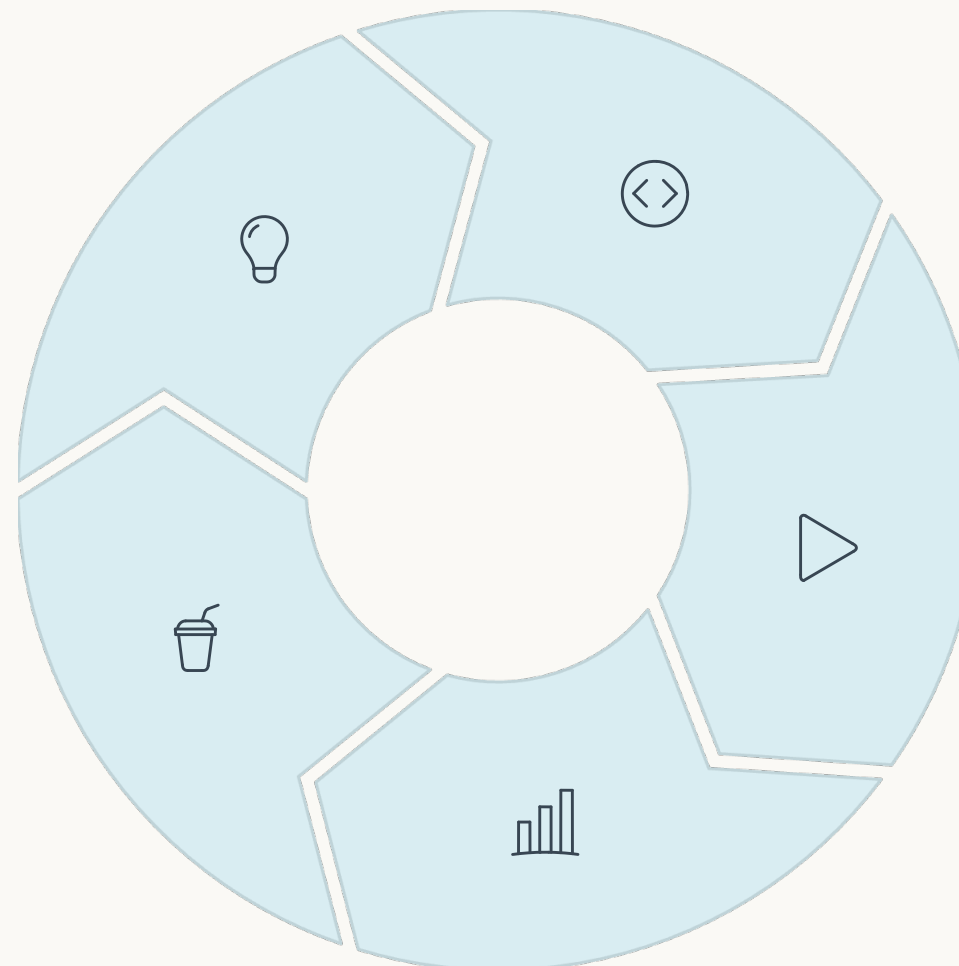
El desarrollo de agentes competitivos es un proceso **iterativo y continuo**, muy común tanto en investigación académica como en aplicaciones industriales. No existe un agente perfecto desde el primer intento.

Diseñar heurísticas

Definir la lógica de decisión del agente.

Modificar la estrategia

Incorporar mejoras y comenzar un nuevo ciclo.



Implementar el agente

Traducir las heurísticas a código funcional.

Ejecutar simulaciones

Lanzar partidas automatizadas contra oponentes.

Analizar resultados

Evaluar métricas y detectar puntos débiles.

Componentes del Sistema

<https://deepwiki.com/jaumejordan/PyCatan>

GameDirector

Orquesta el flujo general del juego: rondas, turnos y fases.

GameManager

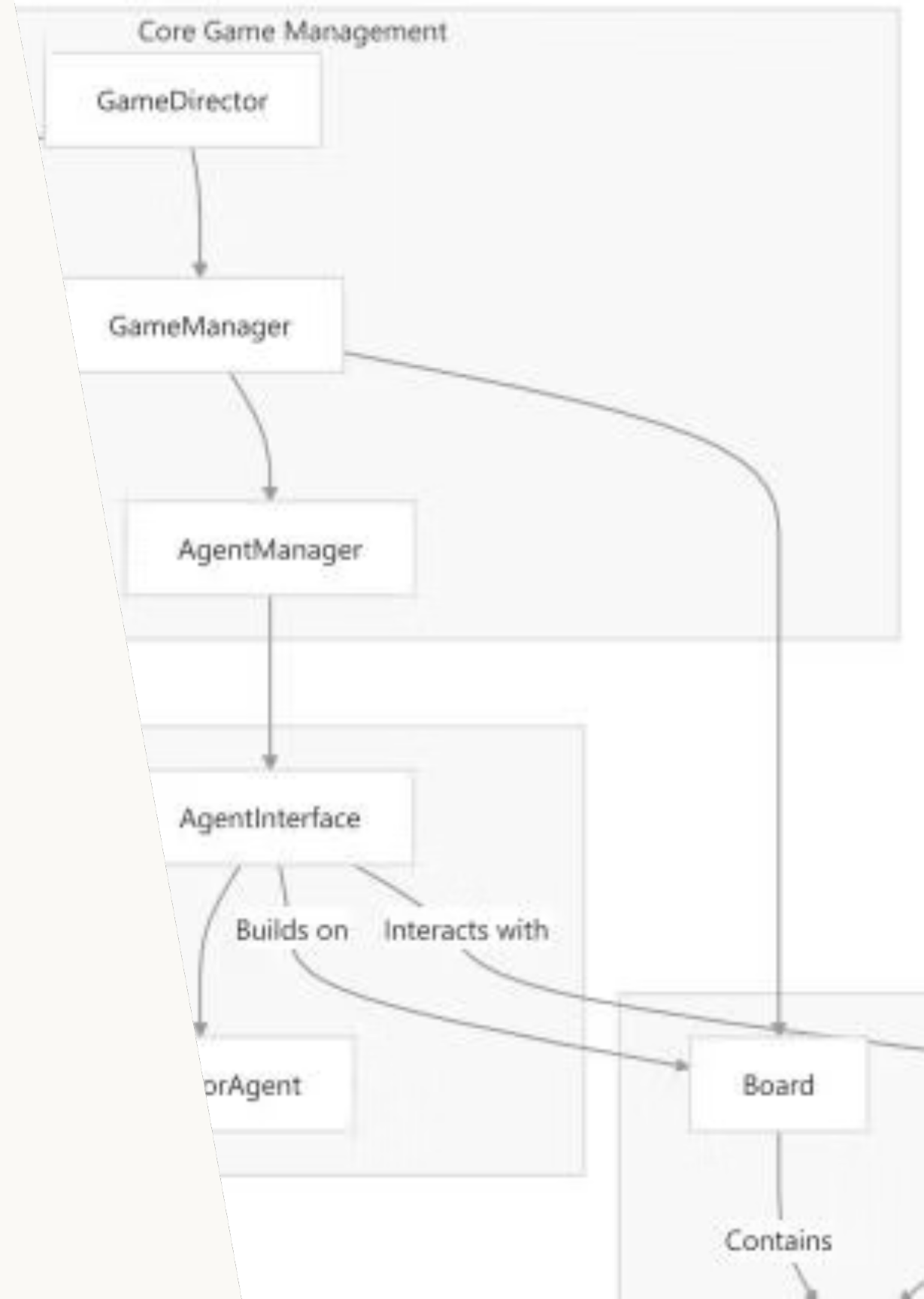
Implementa la mecánica central: recursos, construcción y comercio.

AgentManager

Gestiona la interfaz entre el juego y los agentes de IA.

Board

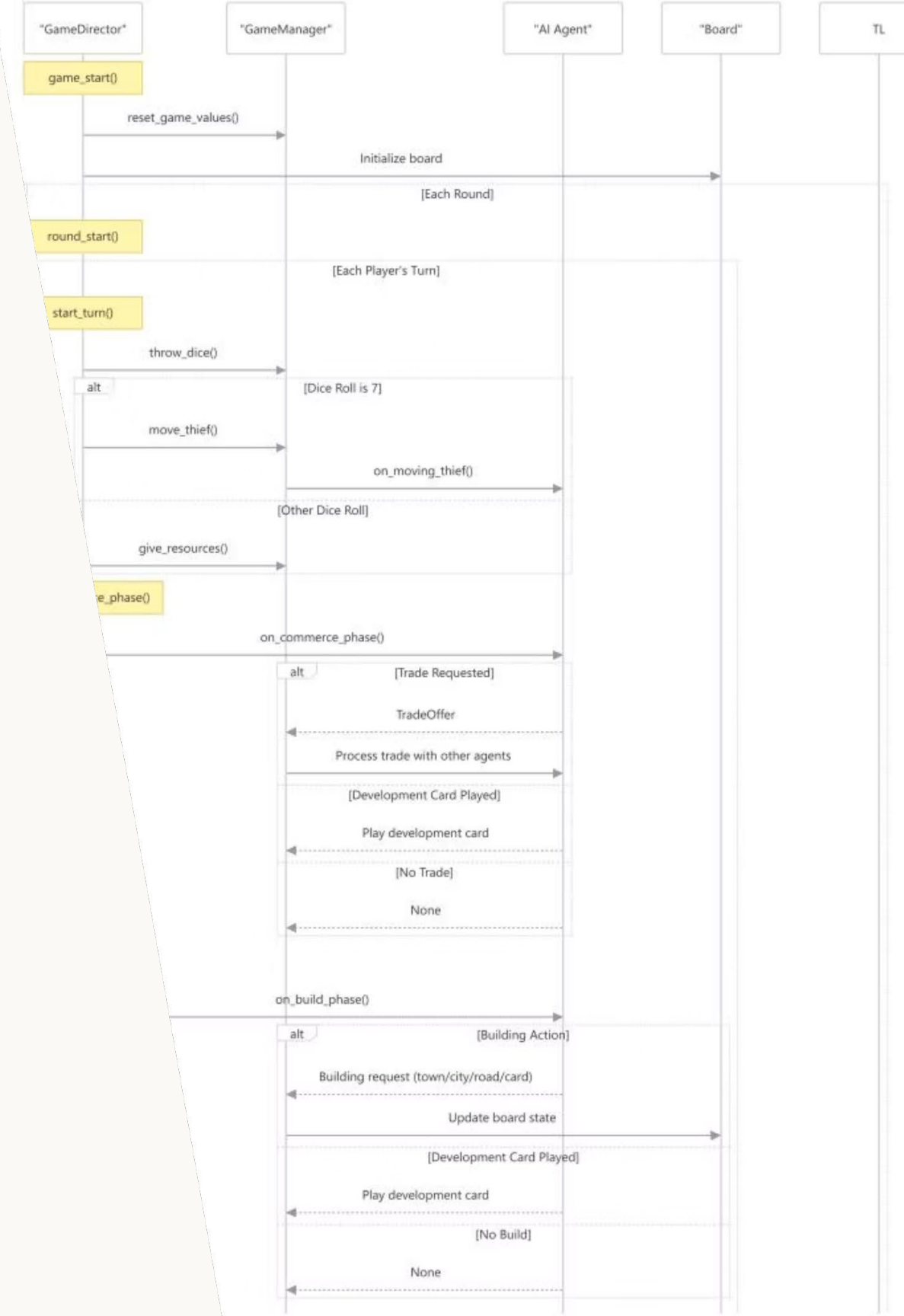
Representa el tablero con terrenos, nodos y caminos.



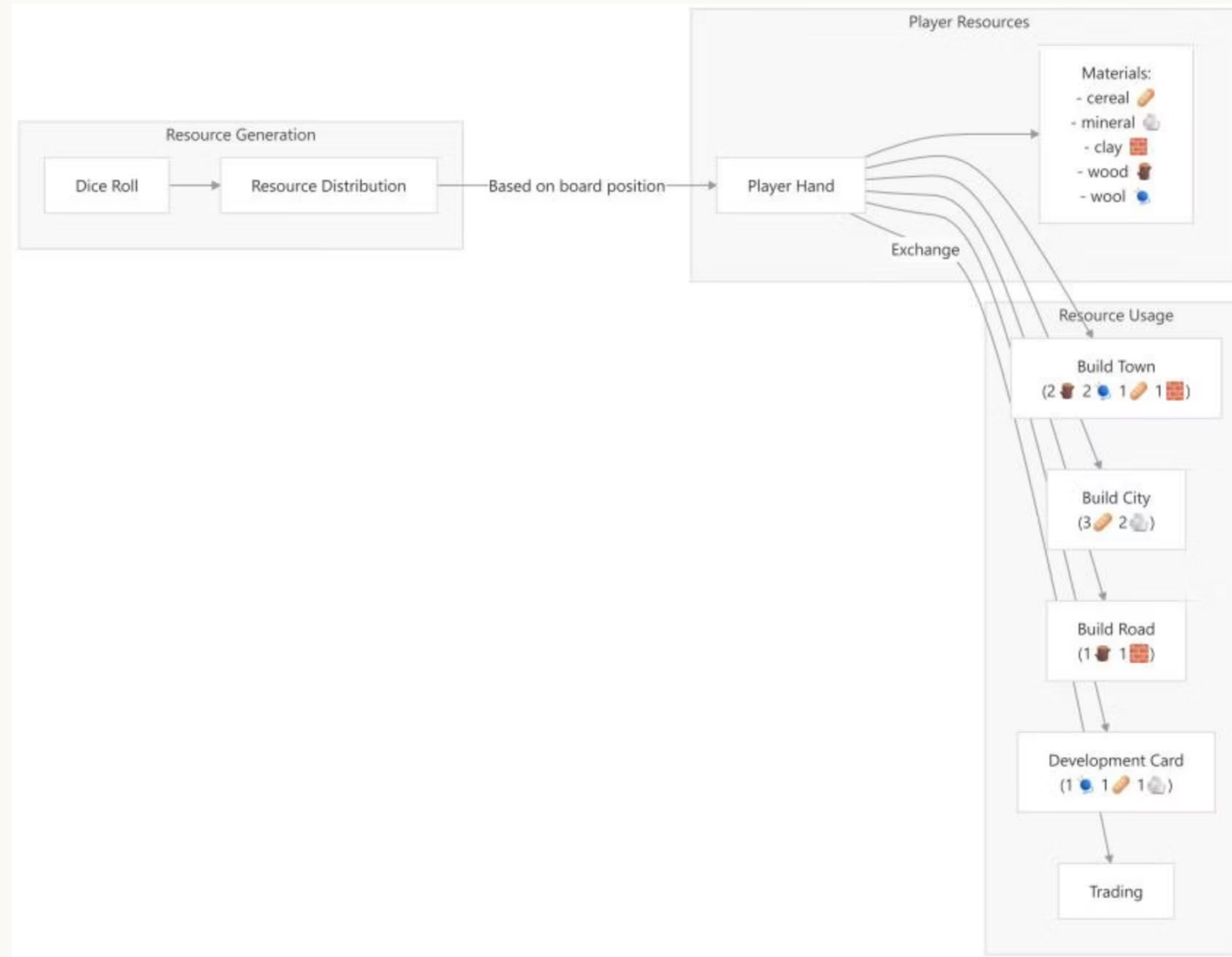
FLUJO DE JUEGO

Secuencia de una Partida

Cada turno sigue tres fases: **inicio de turno** (lanzar dados), **fase de comercio** (intercambiar recursos) y **fase de construcción** (edificar o comprar cartas de desarrollo).



Gestión de Recursos



Los 5 recursos (Materials)

La clase `Materials` soporta suma, resta y comparación de recursos. La clase `Hand` gestiona la mano de cada jugador.

 Cereal

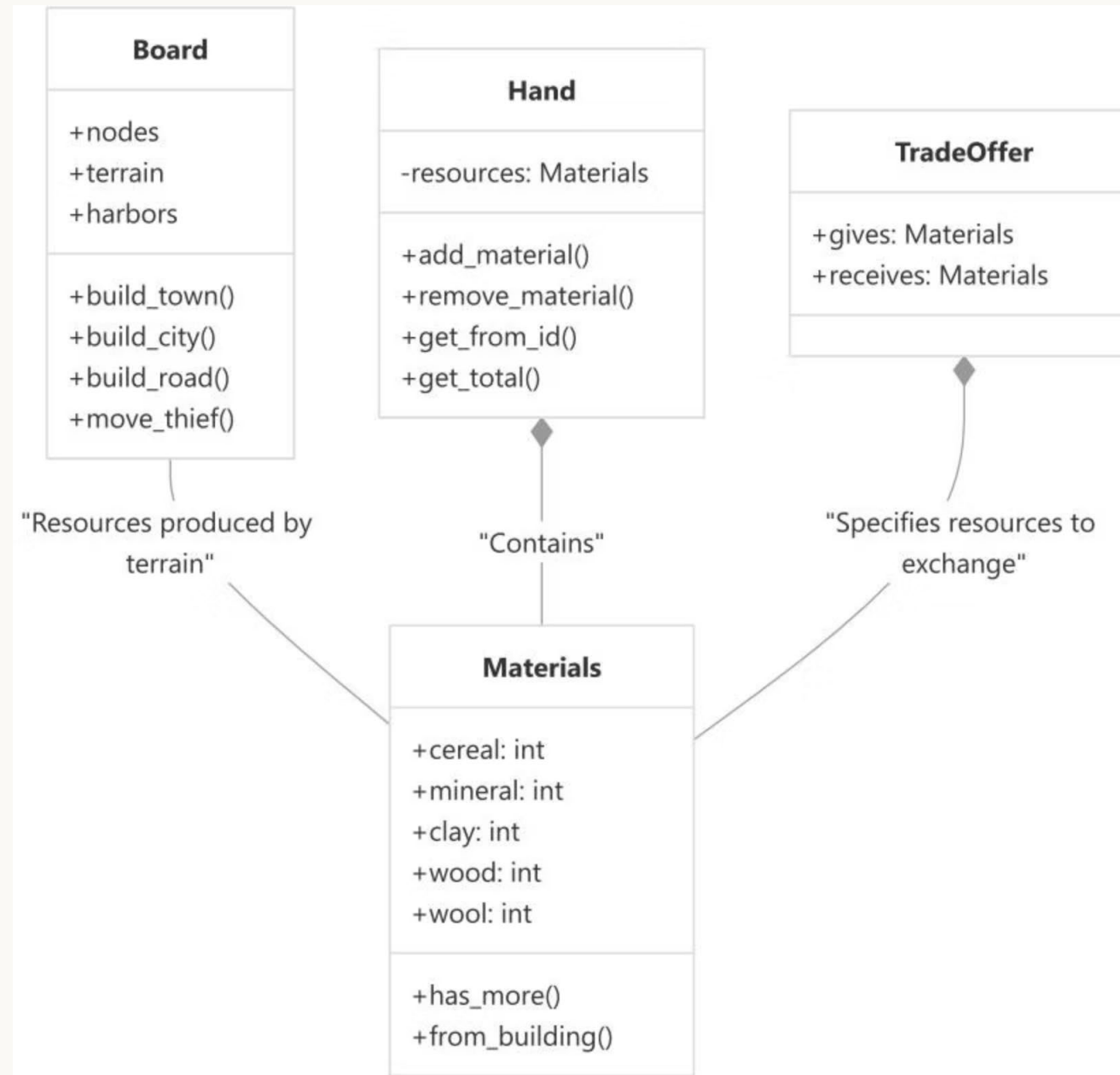
 Mineral

 Arcilla

 Madera

 Lana

Clases del Estado



Relaciones clave

→ Board

54 nodos y 19 hexágonos de terreno.
Gestiona construcciones y el ladrón.

→ Hand

Colección de recursos del jugador. Permite
añadir y retirar materiales.

→ TradeOffer

Define qué recursos se ofrecen (**gives**) y
cuáles se solicitan (**receives**).

Interfaz del Agente (AgentInterface)



Cada método devuelve una acción específica: colocación inicial, carta de desarrollo, oferta de intercambio, acción de construcción o carta al final del turno. Si no se desea actuar, se devuelve **None**.

Ejecutar Simulaciones

```
from Managers.GameDirector import GameDirector
from Agents.RandomAgent import RandomAgent as ra
from Agents.AdrianHerasAgent import AdrianHerasAgent as aha
import random

# Crear director con 4 agentes
game_director = GameDirector(
    agents=(ra, ra, aha, aha),
    max_rounds=200
)

# Ejecutar 100 partidas
for i in range(100):
    random.seed(i)
    game_director.game_start(i, False)
```

Pasos para simular

01

Definir agentes

Seleccionar qué clases de agente participan.

02

Crear GameDirector

Instanciar con agentes y `max_rounds`.

03

Ejecutar partidas

Llamar a `game_start()` con semilla aleatoria.

04

Analizar trazas

Los resultados se guardan como JSON en `Traces/`.

Sistema de Visualización



Board Display

Hexágonos de terreno, asentamientos, caminos y el ladrón en tiempo real.



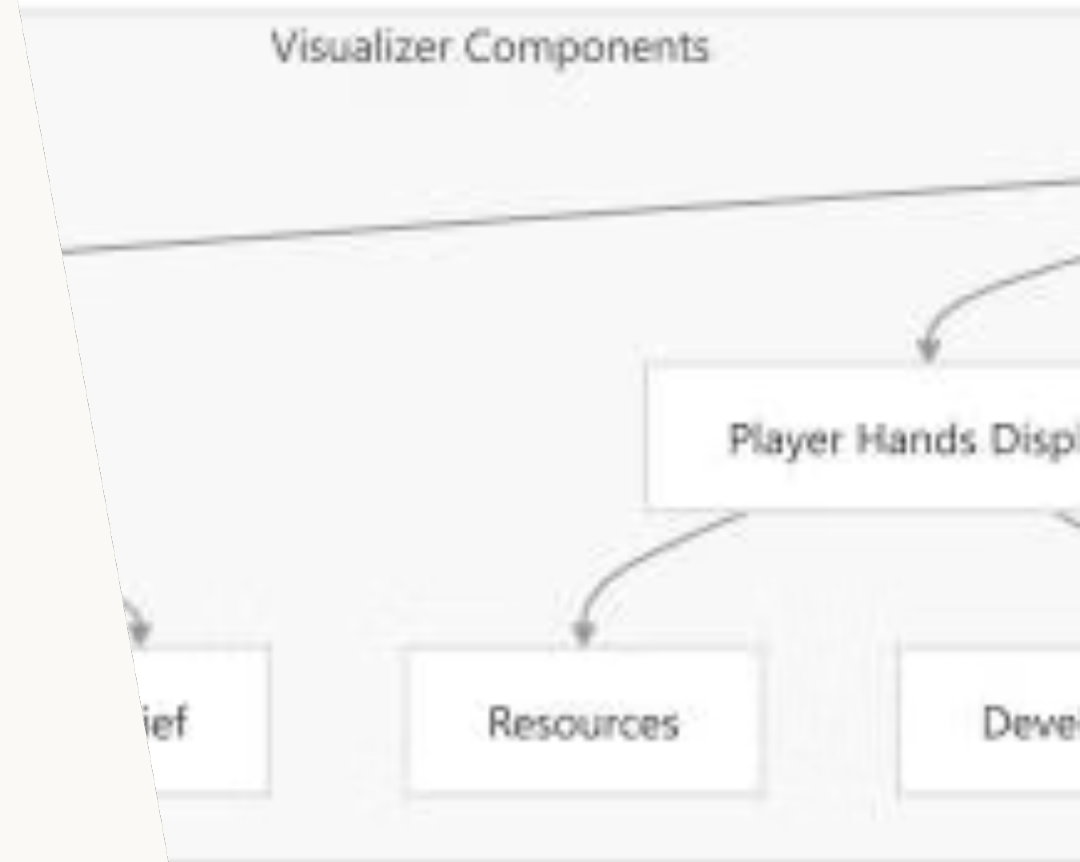
Player Display

Recursos, cartas de desarrollo, ejército más grande y puntos de victoria.



Navegación

Avanza por rondas, turnos y fases. Reproducción automática con control de velocidad.



Mecánica Básica del Juego

1

1. Producción

Lanza dos dados. Los hexágonos con ese número producen recursos para los jugadores adyacentes.

2

2. Comercio

Intercambia con otros jugadores o con el banco (4:1, o mejor ratio con puertos).

3

3. Construcción

Construye asentamientos, carreteras, ciudades o compra cartas de desarrollo.

4

4. Victoria

El primer jugador en alcanzar **10 puntos de victoria** gana la partida.



SIMULADOR

Gestión de Comercio:

on_trade_offer()

Se ejecuta cuando **otro jugador ofrece un intercambio**. El agente evalúa la oferta y decide cómo responder.

```
def on_trade_offer(
    board_instance,
    offer=TradeOffer(),
    player_id=int
):
    # Evalúa la oferta recibida
    # True → acepta
    # False → rechaza
    # TradeOffer → contraoferta
    return True
```

True

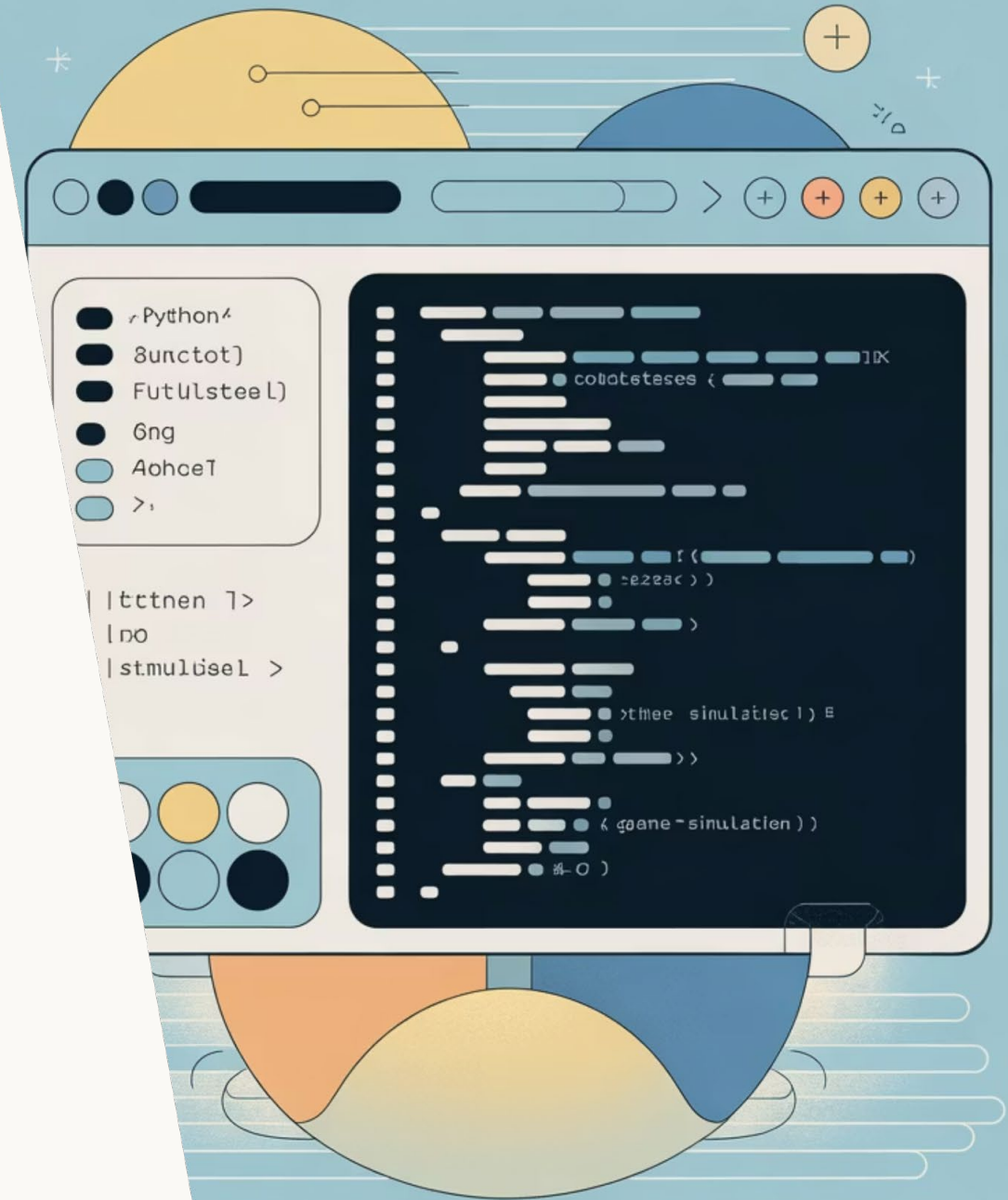
Acepta la oferta

False

Rechaza la oferta

TradeOffer

Devuelve una
contraoferta



Gestión de Comercio: `on_commerce_phase()`

Se ejecuta cuando el agente puede **realizar intercambios de forma activa**: puede ofrecer recursos a otros jugadores o solicitar los que necesita. También para los intercambios con la banca o puertos (se pasa un diccionario).

Esta fase es clave para equilibrar el inventario y acelerar la construcción.

Firma del método

```
def on_commerce_phase():  
    # Decide si ofrecer recursos  
    # TradeOffer → propone cambio  
    # None → pasa sin comerciar  
    return None
```



💡 Usa esta fase para conseguir el recurso que te falta antes de construir.

TradeOffer

Propuesta de intercambio con otro jugador

None

No realiza comercio en este turno

Gestión del Turno: Inicio y Final

on_turn_start()

Se ejecuta **antes de lanzar los dados**. Permite jugar cartas de desarrollo en el momento óptimo.

- **DevelopmentCard** → Juega una carta
- **None** → No usa ninguna carta

on_turn_end()

Se ejecuta **al finalizar el turno**. Última oportunidad para activar una carta de desarrollo antes de pasar el turno.

- **DevelopmentCard** → Juega una carta
- **None** → Finaliza el turno



Solo se puede jugar **una carta de desarrollo** por turno. Elige bien el momento: inicio o final.

SIMULADOR

Gestión de Construcción:

`on_build_phase()`

Se ejecuta durante la **fase de construcción** del turno. El agente decide qué y dónde construir para maximizar su expansión en el tablero.



Carreteras

Expanden el territorio y habilitan nuevas posiciones para asentamientos.



Asentamientos

Generan 1 punto de victoria y producen recursos en los hexágonos adyacentes.



Ciudades

Mejoran un asentamiento existente: 2 puntos de victoria y doble producción.

Retorno: `{'building': str, 'node_id': int, 'road_to': int/None}` o `None` si no construye.



Interacciones con el Ladrón

on_moving_thief()

Se activa al sacar un **7** o al jugar una carta de soldado. El agente elige dónde colocar al ladrón y a quién robarle.

```
return {  
    'terrain': int,    # hexágono  
    'player': int     # jugador objetivo  
}
```

on_having_more_than_7_materials...

Si el agente tiene **más de 7 recursos** cuando se activa el ladrón, debe descartar la mitad.

```
return Hand()  
# Mano con los recursos  
# que el agente descarta
```



 Prioriza descartar recursos que ya tienes en exceso.

Uso de Cartas de Desarrollo

1

`on_monopoly_card_use()`

Elige un tipo de recurso para **tomarlo de todos los jugadores**. Retorna un `int` representando el recurso elegido.

2

`on_road_building_card_use()`

Coloca **dos carreteras gratis**. Retorna un diccionario con `node_id`, `road_to`, `node_id_2` y `road_to_2`.

3

`on_year_of_plenty_card_use()`

Selecciona **dos recursos cualesquiera** del banco. Retorna `{'material': int, 'material_2': int}`.



Inicio del Juego

```
on_game_start(board_instance)
```

Se ejecuta **dos veces** al inicio de la partida. El agente decide la posición estratégica de su primer/segundo asentamiento y la carretera adyacente.

```
return (int, int)
# (nodo_asentamiento,
#  nodo_carretera)
```

- 📋 🏆 Una buena posición inicial es clave para acceder a recursos variados desde el primer turno.



- 📋 Es muy relevante el orden de los jugadores. Normalmente, el primero tiene ventaja al poder elegir la mejor posición inicial

Repositorio, documentación y recogida de resultados

Repositorio: <https://github.com/jaumejordan/PyCatan>

Documentación: <https://deepwiki.com/jaumejordan/PyCatan>



Invocación del simulador con una configuración de agentes predefinidos.



Registro de partidas en un archivo de log o base de datos.

Importante limitar rondas y no guardar la traza si lanzamos muchas partidas para no ocupar espacio

```
try:
    game_director = GameDirector(agents=all_agents, max_rounds=200, store_trace=False)
    game_trace = game_director.game_start(print_outcome=False)
except Exception as e:
    print(f"Error: {e}")
    return 0
```

Extracción del ganador

```
last_round = max(game_trace["game"].keys(), key=lambda r: int(r.split("_")[-1]))
last_turn = max(game_trace["game"][last_round].keys(), key=lambda t: int(t.split("_")[-1].lstrip("P")))
victory_points = game_trace["game"][last_round][last_turn]["end_turn"]["victory_points"]

winner = max(victory_points, key=lambda player: int(victory_points[player]))
fitness = 0
if all_agents.index(chosen_agent) == int(winner.lstrip("J")):
    fitness += 1
```




Resumen para Crear tu Propio Agente

1. Hereda de AgentInterface o RandomAgent

Crea un fichero en `Agents/` e importa `AgentInterface`, `Materials` y `TradeOffer`.

2. Implementa los métodos

Define `on_game_start`, `on_build_phase`, `on_commerce_phase`, `on_trade_offer`, `on_turn_start`, `on_turn_end`, `on_moving_thief` y el resto de eventos. Al principio, que llamen a `super().metodo()`

3. Registra y prueba

Añade tu agente al `GameDirector` y ejecútalo contra los agentes `Random` primero para evaluar su rendimiento.



Trabajo a realizar

El objetivo es construir un agente que **supere consistentemente** al RandomAgent y, progresivamente, a un benchmark de agentes estándar disponible.

01 Implementar vuestro agente heurístico

Diseñad y codificad las reglas de decisión propias

02 Mejorar progresivamente la estrategia

Iterar mediante el ciclo de desarrollo incremental

03 Evaluar mediante simulación

Ejecutar baterías de partidas y analizar métricas

04 Competir contra agentes de referencia

Primero RandomAgent, luego el benchmark completo

05 Uso de LLMs permitido y recomendado

Evaluación de usabilidad de LLMs y herramientas IA (como *opencode*, *antigravity*, *copilot* y cualquier IA generativa). Se debe documentar su uso en la entrega:

- modelos usados
- herramientas
- prompts
- ventajas/desventajas encontradas

En otra sesión se trabajará con detalle con LLMs para la toma de decisiones en PyCatan